Financial Stress Prediction Challenge (September Edition)

Solution documentation, Abdo_VDL (2nd place on the private leaderboard)

Submission	File	Public score	Private score
75o4hiai (the one to verify)	v14_all_MZF_s50_avg5_mrank.csv	0.744624874	0.735747898
QQZviRgL (my second selection)	v14_all_MZF_s50_avg5_mrank_t200.csv	0.744641337	0.735721276

1. Overview and objectives

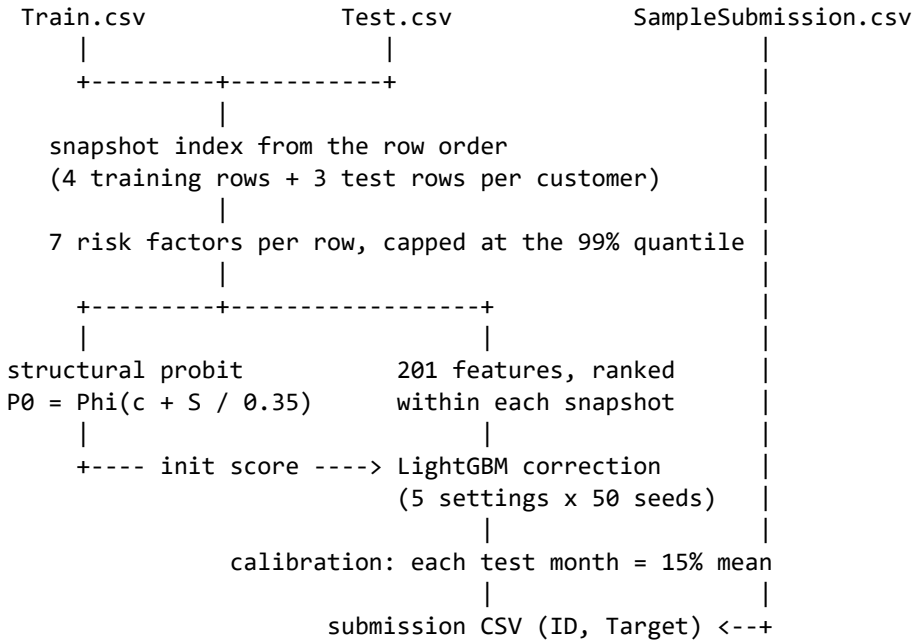
The goal of the challenge is to predict, for each customer snapshot, the probability that the customer will be in liquidity stress during the 30 days after the observation date, using six months of mobile money history. The score is 60% log loss and 40% AUC. So the probabilities have to be well calibrated, and not only well ranked.

I built the solution in two steps:

1. **Structural model.** I compute seven liquidity-risk factors for each row, such as balance drop, income drop and outflow compared with income. I combine them into one risk score and turn that into a probability with a probit function. During my data analysis I found that the labels behave almost exactly like this model, so it is already a strong and easy-to-explain baseline.
2. **LightGBM correction.** A LightGBM model starts from the structural probability (as its initial score) and learns only the small corrections the seven factors miss.

At the end, the predictions of each month are calibrated so that their average is 15%, which is the stress rate in every training month. The whole thing runs on a normal laptop CPU and gives exactly the same file every time.

2. Architecture diagram



3. Environment and data

Machine	My laptop: HP Pavilion 15, Intel Core i7-1255U (10 cores, 12 threads), 16 GB RAM
System	Windows 11, Python 3.12.3, CPU only (no GPU, no cloud)
Libraries	numpy 2.4.6, pandas 2.3.1, scipy 1.16.1, scikit-learn 1.7.1, lightgbm 4.7.0 (requirements.txt)

Data	Only the competition files Train.csv, Test.csv and SampleSubmission.csv; no external data

All the data processing happens inside the scripts. Nothing was prepared by hand or in another tool.

4. How to run it

Put the three competition files in one folder. Names like "Train (1).csv" also work, because the script searches for them. Then run:

```
pip install -r requirements.txt
python best_model_r1mzf.py --seeds 50 --month-scale rank --data-dir <folder>
```

This rebuilds 75o4hiai. It prints the validation result of each held-out month, writes submissions/best_model_r1mzf.csv and prints the SHA-256 of that file. The hash should start with 270ef799f124. If you copy the submissions folder from my zip into the data folder, the script also compares its output with my original file and prints IDENTICAL.

5. Run time

Measured on the laptop above, in a clean folder:

Command	**What it produces**	**Time**
best_model_r1mzf.py --seeds 50 --month-scale rank	75o4hiai (sha 270ef799f124)	17 min 17 s
same command + --round-scale 2.0	QQZviRgL (sha 9bae8f1947b3)	about 27 min
final_model_mz.py --sigma 0.35	structural model alone	about 3 s
data_insights.py	the data checks of section 10	about 6 s

The results are reproducible. LightGBM uses fixed seeds (909, 11, 22, 33, 44, then 1001 to 1045) with deterministic=True, force_row_wise=True and 12 threads. I checked this by unzipping my package into an empty folder with only the competition data, and both selected files came out byte for byte identical. On a different operating system LightGBM can differ in the last decimals.

6. ETL process

Extract. The script reads the three CSV files with pandas. It checks the shapes (40,000 x 184 for train, 30,000 x 183 for test) and checks that the test IDs are in the same order as the sample submission. Each row is one customer at one observation date, with six monthly windows (M1 is the most recent month) and eight profile columns. The volume is small: 70,000 rows and about 60 MB of CSV, read in a few seconds. In production the extraction would run once a month, when a new batch of customer snapshots is available.

Transform.

- *Snapshot index.* The eight profile columns (arpu, age, gender, region, smartphone, segment, earning_pattern, x_90_d_activity_rate) are fixed for a customer, so they identify the 10,000 customers. Each customer has 4 consecutive rows in Train and 3 in Test. The position of a row inside its customer is its snapshot: 0 to 3 for training and 4 to 6 for test. Every training snapshot has exactly 1,500 positives, so the labels were given inside each snapshot. I use this index only to group rows. No feature mixes different snapshots of a customer.
- *Risk factors.* Recent means months M1 to M3 and older means months M4 to M6.

Factor	**Definition**
F0 balance drop	max(1 - (recent balance + 1) / (older balance + 1), 0)
F1 income drop	same, with income = received + deposit + transfer from bank

Factor	Definition
F2 withdrawal ratio	(recent withdrawals + 1) / (older withdrawals + 1)
F3 discretionary spending ratio	same with paybill + merchant pay
F4 outflow / income	(recent outflow + 1) / (recent income + 1)
F5 balance volatility	standard deviation of the six monthly balances
F6 inactivity	1 - x_90_d_activity_rate

  Each factor is capped at its 99% quantile.
- *Features for LightGBM.* These are the 177 numeric columns transformed with sign(x) * log(1 + |x|), the five categorical columns one-hot encoded (17 columns) and the seven capped factors. That makes 201 columns. Each column is replaced by its percentile rank inside its snapshot.

Load. Everything stays in memory, well under 2 GB of RAM. The only file written is the submission CSV. The probabilities are written in full precision, and the script reads the file back to check it.

About the test data. No test labels are used. Some steps use the test features, which are part of the provided data:

- the 99% caps are computed on all 70,000 rows;
- the z-scores and ranks are computed inside each snapshot, the test snapshots too;
- each test month is calibrated to a 15% mean.

In a real deployment this is the same as scoring one monthly batch of customers. The weights, the noise level and the tree counts are fitted on training labels only. I used the public leaderboard only to compare finished submissions.

7. Data modeling

Structural probit. Each capped factor is standardised inside its snapshot (z0 to z6). The risk score is

S = (5 z0 + 3 z1 + 3 z2 + 2 z3 + 4 z4 + 2 z5 + 1 z6) / 20

and the probability is P0 = Phi(c + S / 0.35), where Phi is the normal distribution function. The intercept c is chosen so that the average of P0 in the snapshot is 15%.

This is the same as assuming that latent stress = S + normal noise with standard deviation 0.35, and that the top 15% of each month are labelled as stressed. I got the weights and the 0.35 from probit fits on the training months.

LightGBM correction. LightGBM is trained with init_score = logit(P0), so it only learns what the structural score misses.

Setting	Value
Base parameters	learning_rate 0.02, num_leaves 7, max_depth 3, min_data_in_leaf 300, lambda_l2 50, feature_fraction 0.7, bagging_fraction 0.8
Variants averaged	base; num_leaves 15 + max_depth 4; min_data_in_leaf 100; learning_rate 0.01; learning_rate 0.05
Seeds	50 per variant
Number of trees	early stopping per fold (patience 60, max 600), median over the 4 folds: 26, 25, 18, 55 and 14

For each held-out month, a model is trained on two of the other months and early-stopped on the third. QQZviRgL is the same model with twice the number of trees.

Calibration. The averaged correction is added to logit(P0). Then the intercept of each test month is solved again so that the mean prediction is exactly 15%. The probabilities are written as they are, with no rounding and no thresholds.

Validation. I used leave-one-month-out on the four training snapshots. Each month is held out in turn, and its intercept is computed from the features only. The metric is the competition one, 0.6 x (1 - log loss / 0.595) + 0.4 x AUC. It gives exactly the log loss and AUC shown on the leaderboard.

Model	Validation score	Log loss	AUC
Structural probit alone	0.7404748	0.226702	0.922704
75o4hiai (probit + LightGBM)	0.7405629	0.226641	0.922772
QQZviRgL (twice the trees)	0.7405652	0.226643	0.922781

To check that the gain over the structural model is real, I ran a paired bootstrap on customers with 5,000 resamples. 75o4hiai was better in 93% of them, by 0.000088 on average, and it was better in all four held-out months. Making the LightGBM correction weaker always lowered the score.

8. Inference

The script trains on all labelled snapshots and predicts the test snapshots in the same run, in about 17 minutes. No special infrastructure is needed: it is a batch job that runs on one machine with Python and the five libraries, and it could be scheduled monthly on any server. For a new month of data:

1. Add the new rows, which need the same columns as Test.csv.
2. Compute the caps again over all rows.
3. Rank the features inside the new month.
4. Set the month intercept so that the mean prediction equals the expected stress rate: 15% in this data, or the current observed rate in production.

The output, Target, is a calibrated probability of stress in the next 30 days, so the business can choose any threshold. I would retrain each month when the new labels arrive, and keep the printed validation score and the file hash of each run as its version. With --seeds 10 a run takes about 4 minutes and loses only a little accuracy.

9. Performance metrics

	Public	Private
75o4hiai	0.744624874	0.735747898 (rank 2)
QQZviRgL	0.744641337	0.735721276

While building the solution I mostly followed the leave-one-month-out score, together with its log loss and AUC (section 7). I also used the customer bootstrap above, and the SHA-256 of each file to make sure every rebuild was identical. For the ETL and inference side, the measures are the run times in section 5 (data loading and feature building take a few seconds, the whole run 17 minutes) and the memory use, well under 2 GB.

10. Data insights

data_insights.py reproduces these checks.

- Train and test look the same. The largest Kolmogorov-Smirnov distance between them over all numeric columns is 0.011.
- The strongest single factors are balance drop, income drop and outflow / income, each with an AUC of about 0.72. The other four are much weaker (0.51 to 0.58).
- The structural model seems to match the way the labels were made, because every change I tried fits worse:

Change to the structural model	Effect on the validation score
caps computed on training rows only	-0.000073

Change to the structural model	Effect on the validation score
caps computed per snapshot	-0.000318
caps at 98.9% / 99.1%	-0.000222 / -0.000297
noise 0.33 / 0.37 instead of 0.35	-0.000390 / -0.000275
logistic noise	-0.000703
Student-t noise (5 df)	-0.001543

- The LightGBM corrections mostly follow a few count columns from single months, mainly m5_deposit_agents and the merchant payment columns of months 2 and 3. I tested some variants on the leaderboard:

Variant	Validation vs reference	Public	Private
75o4hiai (reference)	-	0.744625	0.735748
without the profile columns	-0.000005	0.744617	0.735762
z-scores instead of ranks	-0.000007	0.744581	0.735741
twice the trees (QQZviRgL)	+0.000002	0.744641	0.735721
without the four month-5 deposit columns	-0.000072 (vs QQZviRgL)	0.744678	0.735802

  The month-5 deposit pattern is strong in all training months, but it did not hold in the test months: removing those columns was better on both leaderboards. For a production model, I would check this kind of single-month effect again on every new batch.

11. Error handling and logging

The script checks its inputs and stops with an assertion error if:

- a file has the wrong shape;
- the test rows are not in the sample submission order;
- a customer does not have 4 training rows and 3 test rows;
- a training month does not have exactly 1,500 positives;
- a factor value is not finite;
- a probit fit does not converge.

On the output side, every probability must be finite and strictly between 0 and 1. The CSV is read back and compared value by value, and the mean of each test month is printed (it must be 0.15). The SHA-256 is printed too, along with IDENTICAL or DIFFERENT when my original file is present.

During the run the script logs each validation month ("month k done"), the gain per month and the tree counts, so a run can be compared with this document line by line.

12. Maintenance and monitoring

- Each month I would watch the stress rate, the drift of every column against the training months (KS distance) and the 99% caps of the seven factors, and retrain if they move. The stress rate is also the calibration target.
- If the LightGBM correction stops improving the validation score, final_model_mz.py gives the structural model alone. It needs no machine learning library and is easy to explain.
- Corrections that come from one specific month, like the month-5 deposit columns, should be checked again on each new batch.
- The run time grows linearly with the number of rows and seeds. A bigger portfolio only needs more CPU threads, or fewer seeds.

13. Files

File	Content
README.md	this document (also as Documentation_Abdo_VDL.pdf)
requirements.txt	library versions
best_model_r1mzf.py	the full pipeline: data preparation, both models, calibration, validation, submission file
final_model_mz.py	the structural model alone
data_insights.py	the data checks of section 10
submissions/	my original 75o4hiai and QQZviRgL files, used for the IDENTICAL check